Politecnico di Milano

Ingegneria del Software – a.a. 2009/10

Appello del 01 Luglio 2010

Cognome
Nome
Matricola

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h15.
6. Gli studenti che hanno una prova valida devono svolgere il solo Esercizio 3, con consegna in 1h15m. In caso di consegna oltre il termine, la prova precedente perde di validità e si valuterà pertanto l'intero tema.
7. Punteggio totale a disposizione: 100 punti nominali. La sufficienza si raggiunge indicativamente con 60 punti per il tema completo, 30 punti per chi deve svolgere solo l'esercizio 3.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 35)

Si consideri il metodo statico **verificaAnagrammi**. Il metodo riceve come parametro una matrice rettangolare MxN (con N>=2, M>=1) di caratteri. Una matrice, come di consueto in Java, è organizzata come array di array: m[i][j] rappresenta l'elemento sulla riga i, colonna j.
Si noti che ogni riga e colonna della matrice può essere intesa come una stringa (partendo dal carattere in posizione 0, poi in posizione 1, ecc.). Il metodo restituisce true se tutte le seguenti condizioni sono verificate:

- Ogni colonna da 1 a N-1 (intesa come rappresentazione di una stringa a partire dal carattere in posizione 0) è un anagramma della colonna 0.
- Le righe sono in ordine lessicografico crescente a partire dalla riga 0.

a- Si scrivano la pre- e post-condizione del metodo evidenziando tutti gli elementi significativi
Sol:

```
/*@requires m!= null && m.length>=2 &&m[0] != null && m[0].length>=1 &&
@                (\forall int i; 0<=i&&i<m.length; m[i]!=null && m[i].length == m[0].length);
@ ensures \result == (\forall int i; 0<i&&i<m.length;
@                          anagramma(m[0],m[i])) && minoreUguale(m[i-1], m[i]);
@*/
```

dove anagramma e minoreUguale sono funzioni ausiliarie così definite:

```
/*@requires a!=null && b!=null && a.length == b.length;
@ensures \result ==
@       (\forall int i; 0<=i && i<a.length;
@                (\num_of int j; 0<=j && j<=a.length-1;  a[i] == b[j]) ==
@                (\num_of int h; 0<=h && h<a.length;  a[i]==a[h]));
@*/
public static boolean anagramma(char[] a, char[] b)

/*@requires a!=null && b!=null && a.length == b.length;
@ensures \result ==
@       (\exists j; 0<=j && j<=a.length;
@                (\forall int i; 0<=i && i<j; a[i].compareTo(b[i]) =0) &&
                 (j<a.length ==> a[j].compareTo(b[j])<0;
public static boolean minoreUguale(char[] a, char[] b)
```

static boolean verificaAnagrammi(char[][] m)

b- Si identifichino le eccezioni necessarie per rendere totale il metodo e si scrivano le opportune modifiche alla specifica di cui al punto (a).
SOL:
Detta PRE la precondizione di cui al punto a, e POST la postcondizione, si ottiene

```
/*@ ensures \old(PRE) && POST;
@signals (MatriceIllegaleException e) !\old(PRE);
```

Esercizio 2 (punti 15)

Mostrare l'output prodotto dal seguente programma:

```
class Alfa {
	int i;
	int j = 3;
}

abstract class Beta {
	protected int i;
	Beta(int i) { this.i = i; }
	abstract int metodo(Alfa p);
}

class Gamma extends Beta {
	private int j = 1;
	Gamma(int i) { super(i); }
	int metodo(Alfa p) {
		p.i++;
		int v = i + j;
		return v;
	}
}

public class Delta extends Gamma {
	private int k = 3;
	Delta(int i) { super(i); }
	public int metodo(Alfa p) {
		int w = super. metodo(p);
		p.j--;
		return w + k;
	}
	public static void main(String[] args) {
		Delta q = new Delta(6);
		Alfa p = new Alfa();
		System.out.println(q.metodo(p));
		System.out.println(p.i + ", " + p.j);
	}
}
```

RISPOSTA:

SOL:

10

1, 2

Esercizio 3 (punti 50)

Si consideri dapprima il seguente interfaccia Comparable, dalle librerie Java standard:

```
public interface Comparable<T> {
//@ensures (* if this minore di t then \result <0, else if this uguale a t then
//@ \result == 0,  else \result ==1 *)
int  compareTo(T t)
}
```

Il metodo *compareTo* definisce un ordinamento totale sugli elementi del tipo generico T. Si consideri la seguente specifica di una classe generica PriorityQueue, sempre tratta dalle librerie standard Java.

```
public class PriorityQueue<E>{
/* OVERVIEW: Collezione mutabile di oggetti, di tipo generico E in cui gli elementi sono disposti in ordine
dal più piccolo (il primo elemento, ossia la testa) al più grande. Per l'ordinamento si usa il metodo
compareTo di E. Il tipo E deve implementare l'interfaccia Comparable.  */

//@ensures (*inizializza this alla coda vuota*).
public PriorityQueue ()
//osservatori puri:

//@ensures (* \result è la cardinalità di this *)
public /*@ pure @*/ int size()

//@ensures (* \result è true se x compare in this *)
public /*@ pure @*/ boolean contains(Object x)

//@ensures (* \result è l'elemento in posizione i  in this *)
public /*@ pure @*/ E get(int i)

//@ensures (size()>0  ? (\forall E y; contains(y); \result.compareTo(y)<=0)
//@                   : \result == null);
public /*@ pure @*/ E peek()

//mutators:
//@ensures x!= null && (* x è aggiunto a this *);
//@signals (NullPointerException e) x==null;
public void add(E x)  throws NullPointerException

//@ ensures (\old(size()>0)    ? \result == \old(peek()) &&(* \result è eliminato da this *)
//@                            : \result == null && size()==0;
public E poll()
}
```

Si osservi che 1) *add*(x) inserisce l'elemento x nella coda, purche' x!=null; 2) *peek*() restituisce il primo elemento della coda, ossia il più piccolo secondo l'ordinamento definito da E per il metodo compareTo (se la coda è vuota restituisce null); 2) *poll*(), oltre a restituire il primo elemento nello stesso modo di peek, lo rimuove dalla coda.

Domanda A) Scrivere un invariante pubblico per la classe, che catturi le caratteristiche essenziali di PriorityQueue così come descritte nella Overview.

SOL:
public invariant (\forall int i; 0<=i && i<this.size()-1; this.get(i).compareTo(this.get(i+1))<=0;

È data *un'implementazione* di PriorityQueue realizzata nel modo seguente. Un attributo *coda* di tipo Vector memorizza tutti gli elementi della PriorityQueue, dal più grande (in posizione 0) al più piccolo (in ultima posizione). Un attributo *testa* memorizza la testa della coda, ossia l'elemento minimo.
Il rep quindi è:

```
private Vector<E> coda;
private E testa;
```

Si ipotizza che man mano che gli elementi vengono inseriti o tolti dalla coda, il vector *coda* mantenga l'ordine dal più grande al più piccolo.
Sono date le seguenti implementazioni del costruttore:

```
public  PriorityQueue() {
        coda = new Vector<E>;
}
```

e di peek() e di poll():

```
public E peek() {
        if (coda.isEmpty()) throw new EmptyException();
        return testa;
}

public E poll() {
        if (coda.isEmpty()) throw new EmptyException();
        if (coda.size()>=2)
                testa = coda.get(lastElement()-1);
        return coda.removeElementAt(coda.lastElement());
}
```

Domanda B) : Scrivere l'invariante di rappresentazione di PriorityQueue.

SOL
private invariant coda!=null && (coda.size()>0 ==> coda.lastElement()==testa) &&
(\forall int i; 0<i&& i<coda.size();coda.get(i-1).compareTo(coda.get(i))>=0;

Domanda C): Scrivere la funzione di astrazione di PriorityQueue, utilizzando un private invariant.

SOL

```
private invariant (\forall int i; 0<i&& i<coda.size(); coda.get(i)==this.get(size()-i-1);
```

Domanda D): Implementare la funzione di astrazione con toString().

```
public String toString() {
        String s = "";
        (for int i=0; i<coda.size(); s = s+coda.get(coda.size()-i-1);
        return s;
}
```

Si consideri ora una classe DistinctElements-PriorityQueue, che ha gli stessi metodi di PriorityQueue, ma che ridefinisce add in modo non ammettere l'inserimento di elementi duplicati: add non aggiunge l'elemento *x* qualora x.compareTo(y) restituisca 0 per almeno un elemento y in this.

Domanda E): Discutere se è possibile definire DistinctElements-PriorityQueue come estensione di PriorityQueue, *rispettando il principio di sostituzione di Liskov*.

SOL: No, perché per definire la nuova classe occorre ridefinire la specifica di add. Ci sono due soli modi: o si rafforza la precondizione, richiedendo che add(x) non possa essere chiamato con un elemento x già esistente in this; oppure, si indebolisce la postcondizione, non inserendo x in this qualora sia un duplicato. In entrambi i casi, si viola la regola dei metodi.